

Motion and Steering Controller Development Report

Straight-line motion tuning, steering calibration, backlash compensation and deterministic centering

Report date	6 September 2026
System	4-wheel robot; rear-wheel drive with encoder feedback; servo-steered front wheels
Primary validation	Nominal 1000 mm straight motion at 2000 CPS with IMU yaw feedback
Lower-level wheel control	Calibrated feedforward + proportional feedback, $K_p \approx 0.02$, $K_i = 0$
Rear synchronisation	Geometry-aware accumulated travel synchroniser, $K_{sync} = 10$
Current heading controller	$K_p = 0.80$, $K_i = 0$, $K_d = 0$, output limited to ± 0.010 rad
Current steering dynamics	Reversal deadband 0.0005 rad; raw-command slew limit 60 units/s; deterministic centering enabled
Latest validation battery	11.06 V for runs 04–06; 11.05 V for runs 07–09
Engineering decision	Freeze the present straight-line baseline and treat deterministic centering as part of normal steering startup

Purpose

Document the tuning, fixes and implementation changes that transformed the motion controller from independent rear-wheel speed regulation with a simple heading correction into a repeatable straight-line control stack that explicitly handles rear-wheel relationship error, steering hysteresis, backlash traversal, actuator-rate limits and deterministic mechanical startup state.

Milestone result. Across the six latest deterministic-centering runs at $K_p = 0.80$, the mean last-DRIVE yaw error was approximately 0.256° with a sample standard deviation of approximately 0.100° . The observed last-DRIVE range was approximately 0.099° to 0.340° , and none of these six runs entered the earlier long-duration steering-saturation mode.

Executive Summary

The development effort showed that straight-line heading performance was limited by more than the heading-controller gain. Independent rear-wheel speed loops were individually stable yet allowed a small accumulated left-right travel mismatch. A geometry-aware outer synchronisation loop was therefore added in `MotionController` and tuned to $K_{sync} = 10$. Steering calibration then revealed substantial command hysteresis: the same servo command could correspond to different effective steering angles depending on approach direction. This motivated a redesign of `SteeringController` around calibrated effective steering angle rather than raw servo command.

A reversal deadband eliminated very high-frequency branch chatter, but exposed slower physical steering oscillation. The controller was then extended with physically meaningful backlash take-up and rate-limited raw servo motion. Physical testing established 60 raw command units/s as the current safe rate: 65 units/s produced oscillation on its first test, 70 units/s was intermittently oscillatory, and 80 units/s visibly reintroduced oscillation.

Heading proportional gain was subsequently tuned in the new physical-unit architecture. $K_p = 0.75$ clearly improved yaw relative to $K_p = 0.60$, while $K_p = 0.90$ spent excessive time at the ± 0.010 rad steering limit and performed worse. Initial $K_p = 0.80$ runs were highly variable, which led to the identification of a second mechanical-state problem: software centering did not guarantee that the physical linkage started on the same side of its backlash. A deterministic two-stage centering sequence was added so the calibrated center is always approached from a known command direction.

With deterministic centering, $K_p = 0.80$ became both accurate and repeatable. The first three runs at 11.06 V produced last-DRIVE yaw values of approximately 0.301° , 0.340° and 0.161° . A second three-run batch at 11.05 V produced approximately 0.323° , 0.099° and 0.309° . The combined six-run mean was approximately 0.256° with a standard deviation of approximately 0.100° . This constitutes the current straight-line motion-control baseline.

Key findings

Finding	Evidence	Interpretation
Independent wheel loops do not ensure long-horizon synchronisation	Rear-wheel travel mismatch accumulated despite satisfactory local speed regulation	Relative rear-wheel travel requires its own outer control objective
Steering hysteresis is mechanically significant	Focused calibration gave branch-dependent zero commands near +3.85 and -9.53 raw, roughly 13.4 raw units apart	Raw servo command alone is not a sufficient steering state
Rate limiting is required for physical stability	60 raw units/s was smooth; oscillation appeared at 65–80 units/s	Actuator-interface dynamics must be part of controller design
Heading gain has a useful range	$K_p = 0.75$ improved yaw; $K_p = 0.90$ caused extensive output saturation and worse results	More proportional gain is not always more useful authority
Deterministic centering markedly improves repeatability	$K_p = 0.80$ last-DRIVE standard deviation fell from about 0.329° before deterministic centering to about 0.10° over the six latest runs	Mechanical startup branch was a major hidden state variable

1. System Context and Control Structure

The robot has two driven rear wheels with Hall encoders and two front wheels steered through a servo linkage. The software stack is layered so that each controller owns a distinct physical responsibility:

- **DCMotor**: low-level motor actuation and braking.
- **WheelSpeedController**: per-wheel calibrated feedforward plus proportional speed feedback.
- **MotionController**: motion state, distance target, yaw feedback, rear-wheel synchronisation, braking and vehicle kinematics.
- **SteeringController**: calibrated steering-angle interface, branch-dependent inverse calibration, backlash model, reversal logic, raw-command slew limiting and centering behaviour.

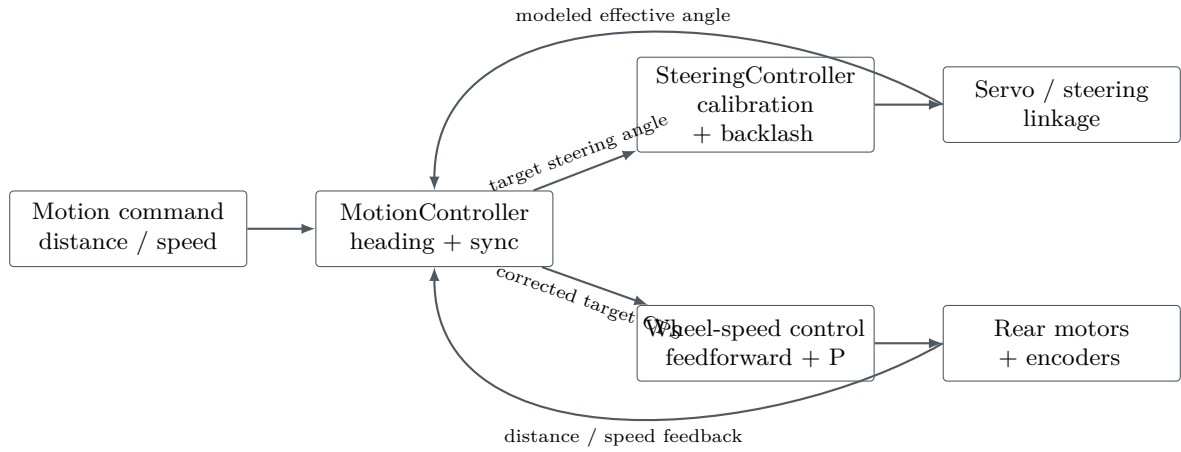


Figure 1: Current control layering. *MotionController* consumes physical steering state and rear-wheel relationship feedback while lower layers retain actuator-specific details.

The retained vehicle geometry is approximately:

Wheelbase L	150.0 mm
Rear track W	165.0 mm
Wheel diameter	65.5 mm
Rear encoder resolution	1560 counts/revolution

The equivalent bicycle-model curvature is

$$\kappa = \frac{\tan \delta}{L},$$

and the nominal rear-wheel velocities are allocated as

$$v_L = v \left(1 - \frac{W}{2} \kappa \right), \quad v_R = v \left(1 + \frac{W}{2} \kappa \right).$$

A critical implementation rule established during testing is that vehicle kinematics must use the current *effective model-estimated* steering angle, not the requested target angle. During backlash take-up the target may already have changed while the wheels have not yet physically changed branch.

2. Rear-Wheel Synchronisation and Distance Control

Earlier straight-line testing showed that satisfactory instantaneous speed regulation did not prevent a small left-right distance mismatch from accumulating over the full motion. The outer synchronisation loop therefore regulates the accumulated relationship error

$$e_{sync} = (d_R - d_L) - \Delta d_{desired}.$$

The symmetric correction is

$$\Delta v_{sync} = K_{sync} e_{sync},$$

with corrected wheel targets

$$v_L^* = v_{L,geom} + \Delta v_{sync}, \quad v_R^* = v_{R,geom} - \Delta v_{sync}.$$

The gain progression $K_{sync} = 0 \rightarrow 5 \rightarrow 10$ produced a clear reduction in accumulated rear-wheel mismatch without sustained target oscillation. The selected value is therefore

$$K_{sync} = 10.0.$$

For straight motion, $\Delta d_{desired} = 0$. For non-zero steering, the desired rear-wheel travel relationship is generated from the vehicle geometry and accumulated using the current effective steering angle.

Distance accuracy was also improved by initiating braking before the nominal distance target. At 2000 CPS, the retained early-braking compensation is approximately 22 mm. The latest runs continue to finish close to 1000 mm average rear-wheel travel while heading control is active.

3. Steering Calibration and Hysteresis

The front steering mechanism has no direct steering-angle sensor. The servo shaft is internally position-controlled, but the linkage between the servo and wheels contains mechanical play outside the servo feedback loop. Consequently, a software model of effective steering angle is required.

Manual rolling calibration was performed at fixed raw steering commands. Using average rear travel

$$s = \frac{d_L + d_R}{2},$$

and gyro yaw change $\Delta\psi$, curvature was estimated as

$$\kappa = \frac{\Delta\psi}{s},$$

with equivalent steering angle

$$\delta = \tan^{-1}(L\kappa).$$

Directional sweeps showed strong hysteresis. The focused sweep near center gave approximate zero-curvature commands of

$$u_{0,inc} \approx +3.85, \quad u_{0,dec} \approx -9.53,$$

a separation of roughly 13.4 raw command units.

Consequence. A single raw servo command does not uniquely determine front-wheel angle. The steering state must also include the direction/branch from which the command was approached.

The common calibrated effective-angle range of both major branches was approximately

$$-0.0516 \leq \delta \leq +0.0161 \text{ rad.}$$

The calibrated zero on the decreasing-command branch, approximately -9.525 raw units, became the preferred software and mechanical center reference for deterministic startup.

4. SteeringController Redesign

The initial steering interface exposed raw servo command directly to higher-level motion control. This was replaced by an effective-angle interface:

```
SteeringController_SetEffectiveAngleRad(...)
```

`MotionController` now requests an equivalent bicycle steering angle, while `SteeringController` owns the inverse calibration, raw command polarity and hysteresis behaviour.

The controller distinguishes three important quantities:

Quantity	Meaning
<code>targetEffectiveAngleRad</code>	Requested steering angle from the heading controller
<code>effectiveAngleRad</code>	Model estimate of the steering angle currently established mechanically
<code>command</code>	Actual raw servo command being applied

The heading loop was also converted to consistent physical units. With yaw measured in degrees,

$$e_\psi = -\psi_{deg} \frac{\pi}{180},$$

and the PID output is interpreted directly as a steering-angle correction in radians. The current heading control remains purely proportional:

$$\delta^* = K_p e_\psi,$$

with $K_i = K_d = 0$ and output saturation at ± 0.010 rad.

5. Reversal Deadband, Backlash Traversal and Rate Limiting

The first target-angle implementation used static inverse calibration at every update. Although algebraically correct, this caused branch selection to switch extremely rapidly when the requested angle fluctuated near a reversal. Recorded raw-direction reversal rates were of the order of tens of reversals per second.

A reversal deadband was introduced:

$$\delta_{deadband} = 0.0005 \text{ rad} \approx 0.0286^\circ.$$

This eliminated the high-frequency numerical chatter, but physical testing revealed a slower visible and audible left-right steering oscillation. The static inverse model was still allowing the commanded raw servo position to jump across the mechanical hysteresis gap too aggressively.

The reversal behaviour was therefore changed to a dynamic state transition:

1. Detect a genuine requested direction reversal outside the reversal deadband.
2. Select the opposite calibration branch.
3. Compute the raw command on that branch corresponding to the *current* effective steering angle.
4. Slew the raw command toward that take-up point.
5. Hold `effectiveAngleRad` at the previous value while the linkage traverses its backlash.
6. Once the take-up boundary is reached, clear backlash state and resume normal target tracking on the new branch.

The raw command is slew-limited according to

$$|\Delta u| \leq \dot{u}_{max} \Delta t.$$

Physical rate testing gave the following boundary:

Maximum command rate	Physical observation
40 units/s	Smooth, but steering response noticeably too slow
60 units/s	No visible or audible oscillation in tested runs
65 units/s	Oscillation observed on the first run
70 units/s	Intermittent mechanical oscillation
80 units/s	Mechanical oscillation clearly begins to appear

The selected actuator-interface limit is therefore

$$\dot{u}_{max} = 60 \text{ raw command units/s}.$$

This is an experimentally selected safe rate for the complete servo/linkage system, not a servo datasheet speed specification.

A startup latch bug was also fixed after the first rate-limited test. The controller could begin with `backlashActive = true` while already positioned at the computed take-up boundary, leaving the model permanently latched. The fix added a generic take-up completion condition and explicit initialization of the known centered state.

6. Heading Gain Tuning

Once steering reversal dynamics were physically credible, the proportional heading gain could be tuned meaningfully while keeping $K_i = K_d = 0$, $K_{sync} = 10$, steering limit 0.010 rad, deadband 0.0005 rad, and rate limit 60 units/s fixed.

Heading K_p	Representative result	Interpretation
0.60	Three 60-units/s runs gave mean last-DRIVE yaw about 0.97°	Stable baseline, but insufficient correction
0.75	Mean last-DRIVE yaw about 0.60° in the initial three-run batch	Clear improvement with no obvious underdamped heading oscillation
0.90	Mean last-DRIVE yaw about 0.77° ; target spent much of DRIVE at ± 0.010 rad	Excessive gain for the current output limit; modulation lost through saturation
0.80 before deterministic centering	Last-DRIVE yaw approximately $0.435^\circ, 0.715^\circ, 1.091^\circ$; $s \approx 0.329^\circ$	High run-to-run variation indicated another state-dependent problem rather than a simple gain optimum

The $K_p = 0.90$ result was especially informative. With a 0.010 rad output limit, saturation begins at approximately

$$|e_\psi| = \frac{0.010}{0.90} \approx 0.0111 \text{ rad} \approx 0.637^\circ.$$

Once saturated, additional proportional gain cannot produce more steering authority; it only causes the controller to reach the limit earlier.

The unexpectedly high variance at $K_p = 0.80$ led to inspection of the steering startup condition rather than further gain changes.

7. Deterministic Mechanical Centering

Software centering originally established a known model state, but did not guarantee that the physical linkage had approached center from a known side of its backlash. Two runs could therefore start with identical software state and servo command while the front-wheel linkage was loaded differently.

A deterministic two-step centering sequence was added to `SteeringController`. Conceptually,

known preconditioning command $\rightarrow$ calibrated center from known direction $\rightarrow$ motion start.

For the selected decreasing-command center branch, the sequence is approximately

$$+12 \text{ raw} \rightarrow -9.525 \text{ raw} \rightarrow \delta_{eff} = 0.$$

The approach uses the same 60-units/s rate limit. After reaching center, the software model is synchronized to the known branch and normal motion begins.

Engineering interpretation. Deterministic centering does not merely set a variable to zero. It physically takes up the steering linkage clearance in a repeatable direction, reducing an otherwise hidden mechanical initial condition.

8. Validation of Deterministic Centering at $K_p = 0.80$

The first deterministic-centering validation batch was performed at a measured battery voltage of 11.06 V. The second batch was performed at 11.05 V. All six runs retained $K_p = 0.80$, $K_i = K_d = 0$, $K_{sync} = 10$, heading limit 0.010 rad, reversal deadband 0.0005 rad, and maximum command rate 60 units/s.

	Run 04	Run 05	Run 06	Run 07	Run 08	Run 09
Battery (V)	11.06	11.06	11.06	11.05	11.05	11.05
Last-DRIVE yaw (deg)	0.301	0.340	0.161	0.323	0.099	0.309
Final yaw (deg)	0.300	0.349	0.147	0.192	0.066	0.339
Mean $ yaw $, DRIVE (deg)	0.143	0.243	0.182	0.340	0.154	0.088
RMS yaw, DRIVE (deg)	0.167	0.267	0.209	0.367	0.170	0.113
Peak $ yaw $, DRIVE (deg)	0.334	0.441	0.399	0.560	0.308	0.331
Steering saturation	0%	0%	0%	0%	0%	0%

The first three deterministic-centering runs gave

$$\bar{\psi}_{last} \approx 0.268^\circ, \quad s \approx 0.094^\circ.$$

The second three-run batch gave

$$\bar{\psi}_{last} \approx 0.243^\circ, \quad s \approx 0.126^\circ.$$

Across all six deterministic-centering runs,

$$\boxed{\bar{\psi}_{last} \approx 0.256^\circ}, \quad \boxed{s \approx 0.100^\circ}.$$

The observed last-DRIVE range was approximately

$$0.099^\circ \leq |\psi_{last}| \leq 0.340^\circ.$$

Before deterministic centering, the three $K_p = 0.80$ last-DRIVE values were approximately 0.435° , 0.715° and 1.091° , with sample standard deviation about 0.329° . The new six-run standard deviation of about 0.100° is therefore substantially smaller, and the earlier long-duration steering-saturation mode did not reappear.

Run 08 achieved the best endpoint result to date, with final yaw approximately 0.066° . Run 09 was notable for its whole-drive behaviour: mean absolute yaw was approximately 0.088° , despite not having the smallest endpoint error. This reinforces the use of whole-trajectory metrics rather than best-case final yaw alone.

9. Current Frozen Configuration

Parameter / behaviour	Current value
WheelSpeedController proportional gain	$K_p \approx 0.02$ for both rear wheels
Rear-wheel synchronisation gain	$K_{sync} = 10.0$
Heading proportional gain	$K_p = 0.80$
Heading integral / derivative gains	$K_i = 0$, $K_d = 0$
Heading steering limit	± 0.010 rad
Steering reversal deadband	0.0005 rad
Maximum raw command rate	60 units/s
Early braking compensation at 2000 CPS	approximately 22 mm
Centering	deterministic two-step physical branch conditioning

These values should now be treated as the current straight-line baseline. Future changes should be made one variable at a time and compared against this configuration.

10. Engineering Lessons and Current Status

The work established several design principles that should be retained as the project moves to more complex motion:

- **Separate physical objectives by layer.** Absolute wheel-speed tracking and relative rear-wheel synchronisation are distinct control problems.
- **Use physical units at abstraction boundaries.** MotionController should request steering angle, not raw servo command.
- **Treat hysteresis as state.** A raw command is insufficient to describe steering when the mechanical response is branch dependent.
- **Model actuator dynamics.** Backlash take-up and finite command slew materially change closed-loop behaviour.
- **Use effective state in kinematics.** Requested target angle must not be substituted for current effective steering during backlash traversal.
- **Physical observation outranks a convenient log proxy.** Faster raw command rates improved some numerical yaw metrics but physically reintroduced steering oscillation.
- **Mechanical initialization can be part of control design.** Deterministic centering converted a highly variable $K_p = 0.80$ result into a substantially tighter six-run distribution.

Current status. The project has progressed beyond basic straight-line controller stabilization. The latest six deterministic-centering runs demonstrate approximately 0.26° mean last-DRIVE yaw error with approximately 0.10° run-to-run standard deviation over a nominal 1 m motion. This is an appropriate baseline from which to proceed to broader motion validation rather than continuing to repair fundamental steering behaviour.

Data Sources and Provenance

- Steering calibration sweeps performed 3 September 2026, including broad and focused directional sweeps used to derive the branch-dependent effective-angle maps and zero commands.
- MotionController integration and heading-control runs performed 4–6 September 2026, including rate-limit sweeps at 40, 60, 65, 70 and 80 raw units/s and proportional heading-gain experiments at $K_p = 0.60, 0.75, 0.80, 0.90$.
- Deterministic-centering validation files: `mctrl_run04.txt`, `mctrl_run05.txt`, `mctrl_run06.txt`, `mctrl_run07.txt`, `mctrl_run08.txt`, and `mctrl_run09.txt`.
- Measured battery voltage for runs 04–06: 11.06 V. Measured battery voltage for runs 07–09: 11.05 V.
- Earlier wheel-synchronisation investigation and controller calibration results used to establish the retained wheel-speed gain, synchronisation architecture, geometry and braking compensation.

Note: reported experiment values are rounded for readability. Best-case endpoint yaw is not treated as the primary performance statistic; whole-drive error and repeatability are emphasized.